

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2
“Модульное тестирование”

Выполнил:
Студент группы ИУ5-33Б
Алиева Э. Р.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Листинг программы

main.py

```
from typing import List, Any, Callable
from dataclasses import dataclass
from operator import itemgetter

@dataclass
class Computer:
    """Класс для представления компьютера"""
    id: int
    model: str
    processor: str
    ram_gb: int

@dataclass
class Browser:
    """Класс для представления браузера"""
    id: int
    name: str
    version: str
    memory_usage: int
    computer_id: int

@dataclass
class ComputerBrowser:
    """Класс для реализации отношения многие ко многим"""
    computer_id: int
    browser_id: int

def get_computers() -> List[Computer]:
    """Функция для генерации данных компьютеров"""
    return [
        Computer(1, 'Dell XPS 15', 'Intel i7', 16),
        Computer(2, 'HP Pavilion', 'AMD Ryzen 5', 8),
        Computer(3, 'Lenovo ThinkPad', 'Intel i5', 16),
        Computer(4, 'Apple MacBook Pro', 'M1 Pro', 32),
        Computer(5, 'ASUS ROG', 'Intel i9', 64),
    ]

def get_browsers() -> List[Browser]:
    """Функция для генерации данных браузеров"""
    return [
        Browser(1, 'Chrome', '120.0', 512, 1),
        Browser(2, 'Firefox', '115.0', 256, 2),
        Browser(3, 'Edge', '119.0', 384, 3),
        Browser(4, 'Safari', '17.0', 128, 4),
        Browser(5, 'Opera', '105.0', 192, 5),
        Browser(6, 'Chrome', '121.0', 520, 2),
        Browser(7, 'Firefox', '116.0', 265, 3),
        Browser(8, 'Arc', '1.0', 320, 1),
    ]
```

```

]

def get_computer_browsers() -> List[ComputerBrowser]:
    """Функция для генерации связей многие ко многим"""
    return [
        ComputerBrowser(1, 1),
        ComputerBrowser(1, 8),
        ComputerBrowser(2, 2),
        ComputerBrowser(2, 6),
        ComputerBrowser(3, 3),
        ComputerBrowser(3, 7),
        ComputerBrowser(4, 4),
        ComputerBrowser(5, 5),
        ComputerBrowser(1, 2),
        ComputerBrowser(3, 1),
    ]

def print_data(data: List[Any], headers: List[str], title: str, column_width: int =
20) -> None:
    total_length = len(headers) * column_width
    columns = len(headers)

    print(f"{{title:=^{{total_length}}}}")
    # ИСПРАВЛЕНИЕ: правильное использование format внутри print
    print(((("{:<" + str(column_width) + "}") * columns).format(*headers))
    print()

    for row in data:
        if isinstance(row, tuple):
            print(((("{:<" + str(column_width) + "}") * columns).format(*row))
        else:
            print(((("{:<" + str(column_width) + "}") * columns).format(row))
    print()

def first_query(computers: List[Computer], browsers: List[Browser]) -> List[Any]:
    """Реализация первого запроса: браузеры, начинающиеся на 'A'"""
    result = list()
    for computer in computers:
        for browser in browsers:
            if computer.id == browser.computer_id and browser.name.startswith('A'):
                result.append((browser.name, browser.version, browser.memory_usage,
                    computer.model, computer.processor))

    result.sort(key=itemgetter(0)) # Сортировка по названию браузера
    return result

def second_query(computers: List[Computer], browsers: List[Browser]) -> List[Any]:
    computer_memory: dict[str, list[int]] = {}

    for computer in computers:
        for browser in browsers:
            if computer.id == browser.computer_id:
                if computer.model not in computer_memory:

```

```

        computer_memory[computer.model] = []
        computer_memory[computer.model].append(browser.memory_usage)

    result = list()
    for computer_model, memories in computer_memory.items():
        min_memory = min(memories)
        result.append((computer_model, min_memory))

    result.sort(key=itemgetter(1)) # Сортировка по минимальной памяти
    return result

def third_query(computers: List[Computer], browsers: List[Browser],
                relations: List[ComputerBrowser], condition: Callable) -> List[Any]:
    result = list()

    for relation in relations:
        computer = next((c for c in computers if c.id == relation.computer_id), None)
        browser = next((b for b in browsers if b.id == relation.browser_id), None)

        if computer and browser and condition(computer.model):
            result.append((browser.name, browser.version, browser.memory_usage,
                           computer.model, computer.processor))

    result.sort(key=itemgetter(0)) # Сортировка по названию браузера
    return result

def main() -> None:
    computers = get_computers()
    browsers = get_browsers()
    computer_browsers = get_computer_browsers()

    # Первый запрос
    print_data(
        first_query(computers, browsers),
        ["Браузер", "Версия", "Память (МБ)", "Компьютер", "Процессор"],
        "Запрос 1: Браузеры, начинающиеся на 'A'",
    )

    # Второй запрос
    print_data(
        second_query(computers, browsers),
        ["Компьютер", "Мин. память (МБ)"],
        "Запрос 2: Минимальная память браузеров"
    )

    # Третий запрос
    print_data(
        third_query(computers, browsers, computer_browsers,
                    lambda model: "Dell" in model or "Apple" in model),
        ["Браузер", "Версия", "Память", "Компьютер", "Процессор"],
        "Запрос 3: Браузеры на Dell и Apple компьютерах"
    )

```

```
if __name__ == "__main__":
    main()
```

test_main.py

```
import pytest
from pytest_unordered import unordered

from main import (
    Computer, Browser, ComputerBrowser,
    first_query, second_query, third_query
)

@pytest.fixture
def test_one_to_many_data():
    """Фикстура для тестов один-ко-многим"""
    computers = [
        Computer(1, 'Dell XPS 15', 'Intel i7', 16),
        Computer(2, 'HP Pavilion', 'AMD Ryzen 5', 8),
        Computer(3, 'Lenovo ThinkPad', 'Intel i5', 16),
    ]

    browsers = [
        Browser(1, 'Arc', '1.0', 320, 1),      # Dell XPS 15
        Browser(2, 'Chrome', '120.0', 512, 1), # Dell XPS 15
        Browser(3, 'Firefox', '115.0', 256, 2), # HP Pavilion
        Browser(4, 'Alphabet', '1.0', 128, 3), # Lenovo ThinkPad
        Browser(5, 'Opera', '105.0', 192, 2),  # HP Pavilion
    ]

    return computers, browsers

@pytest.fixture
def test_many_to_many_data():
    """Фикстура для тестов многие-ко-многим"""
    computers = [
        Computer(1, 'Dell XPS 15', 'Intel i7', 16),
        Computer(2, 'HP Pavilion', 'AMD Ryzen 5', 8),
        Computer(3, 'Apple MacBook', 'M1', 16),
        Computer(4, 'ASUS ROG', 'Intel i9', 32),
    ]

    browsers = [
        Browser(1, 'Chrome', '120.0', 512, 0),
        Browser(2, 'Firefox', '115.0', 256, 0),
        Browser(3, 'Safari', '17.0', 128, 0),
        Browser(4, 'Edge', '119.0', 384, 0),
        Browser(5, 'Opera', '105.0', 192, 0),
    ]

    relations = [
```

```

        ComputerBrowser(1, 1), # Dell - Chrome
        ComputerBrowser(1, 2), # Dell - Firefox
        ComputerBrowser(2, 2), # HP - Firefox
        ComputerBrowser(2, 4), # HP - Edge
        ComputerBrowser(3, 3), # Apple - Safari
        ComputerBrowser(3, 1), # Apple - Chrome
        ComputerBrowser(4, 5), # ASUS - Opera
        ComputerBrowser(4, 4), # ASUS - Edge
    ]

    return computers, browsers, relations

def test_first_query(test_one_to_many_data):
    """Тест первого запроса: браузеры, начинающиеся на 'A'"""
    computers, browsers = test_one_to_many_data

    expected = [
        ('Alphabet', '1.0', 128, 'Lenovo ThinkPad', 'Intel i5'),
        ('Arc', '1.0', 320, 'Dell XPS 15', 'Intel i7'),
    ]

    result = first_query(computers, browsers)

    assert len(result) == 2
    assert result == expected
    assert result[0][0] == 'Alphabet' # Проверка сортировки по алфавиту
    assert result[1][0] == 'Arc'

def test_second_query(test_one_to_many_data):
    """Тест второго запроса: минимальная память браузеров на каждом компьютере"""
    computers, browsers = test_one_to_many_data

    # Правильные ожидаемые значения:
    # 1. Lenovo ThinkPad: только Alphabet (128 МБ) → min = 128
    # 2. HP Pavilion: Firefox (256 МБ) и Opera (192 МБ) → min = 192
    # 3. Dell XPS 15: Arc (320 МБ) и Chrome (512 МБ) → min = 320
    expected = [
        ('Lenovo ThinkPad', 128),
        ('HP Pavilion', 192),
        ('Dell XPS 15', 320),
    ]

    result = second_query(computers, browsers)

    assert len(result) == 3
    assert result == expected
    assert result[0][1] == 128 # Наименьшая минимальная память
    assert result[2][1] == 320 # Наибольшая минимальная память

def test_third_query(test_many_to_many_data):
    """Тест третьего запроса: связь многие ко многим с условием"""
    computers, browsers, relations = test_many_to_many_data

```

```

# Ожидаемый результат для компьютеров Dell и Apple
expected = unordered([
    ('Chrome', '120.0', 512, 'Apple MacBook', 'M1'),
    ('Chrome', '120.0', 512, 'Dell XPS 15', 'Intel i7'),
    ('Firefox', '115.0', 256, 'Dell XPS 15', 'Intel i7'),
    ('Safari', '17.0', 128, 'Apple MacBook', 'M1'),
])

result = third_query(computers, browsers, relations,
                     lambda model: "Dell" in model or "Apple" in model)

assert len(result) == 4

for item in result:
    assert "Dell" in item[3] or "Apple" in item[3]

# Проверка сортировки по названию браузера
browser_names = [item[0] for item in result]
sorted_browser_names = sorted(browser_names)
assert browser_names == sorted_browser_names

def test_first_query_empty_result():
    """Тест первого запроса: когда нет браузеров на 'A'"""
    computers = [Computer(1, 'Test', 'CPU', 8)]
    browsers = [Browser(1, 'Chrome', '1.0', 100, 1)]

    result = first_query(computers, browsers)

    assert len(result) == 0
    assert result == []

def test_second_query_single_computer():
    """Тест второго запроса: один компьютер с несколькими браузерами"""
    computers = [Computer(1, 'Test PC', 'CPU', 8)]
    browsers = [
        Browser(1, 'Browser1', '1.0', 100, 1),
        Browser(2, 'Browser2', '1.0', 200, 1),
        Browser(3, 'Browser3', '1.0', 50, 1),
    ]

    result = second_query(computers, browsers)

    assert len(result) == 1
    assert result[0] == ('Test PC', 50)

```

Результат выполнения программы:

```
(rk2) emiliaalieva@MacBook-Air-Emilia Alieva-PCPL-2025 % /Users/emiliaalieva/Alieva-PCPL-2025/rk2/bin/python /Users/emiliaalieva/Alieva-PCPL-2025/main.py
=====Запрос 1: Браузеры, начинающиеся на 'A'=====
Браузер          Версия          Память (МБ)          Компьютер          Процессор
Arc              1.0              320                  Dell XPS 15        Intel i7

=Запрос 2: Минимальная память браузеров=
Компьютер          Мин. память (МБ)
Apple MacBook Pro  128
ASUS ROG           192
HP Pavilion        256
Lenovo ThinkPad    265
Dell XPS 15        320

=====Запрос 3: Браузеры на Dell и Apple компьютерах=====
Браузер          Версия          Память          Компьютер          Процессор
Arc              1.0              320              Dell XPS 15        Intel i7
Chrome           120.0            512              Dell XPS 15        Intel i7
Firefox           115.0            256              Dell XPS 15        Intel i7
Safari            17.0             128              Apple MacBook Pro  M1 Pro
```

```
(rk2) emiliaalieva@MacBook-Air-Emilia Alieva-PCPL-2025 % pytest
===== test session starts =====
platform darwin -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/emiliaalieva/Alieva-PCPL-2025
plugins: unordered-0.7.0
collected 5 items

test_main.py ..... [100%]

===== 5 passed in 0.02s =====
```